DATE 13/10/26 CUDA

S M T W T F S
O O ● O O O O

In Java, we have studied how to create byte variable & array.
Here, instead of creating byte variable, we create :

unsigned char → ranges from 0 to 255 (unsigned)

So, say we need to increase Brightness of a picture, so we need to add every pixel with 40 so that its intensity ↑
Here, we use unsigned char because, C++ int takes up 4 bytes of space whereas, unsigned char takes 1 byte. (8 bits)

Therefore, Memory needed ↓

We create

unsigned char * d_x
unsigned char image[12] = {10, 50, ...}

All other process same & also as we know byte value can unsigned char can hold

DATE / /

S M T W T F S
O O O O O O O

only values from 0 to 255, so, our o/p value inside array, must not exceed 255.

∴ int newval = img[index] + 40;
if (newval > 255) {
newval = 255;
}
img[index] = newval;

GPU Memory Hierarchy

When we are performing operations on the various kernels, we need memory to access the data from.

Ex) We use cudaMalloc to create a Memory space in GPU, which could be accessed by all the threads currently running.

DATE / / S M T W T F S

(i) Global Memory:

the memory space where we copy the data from CPU to GPU, threads can access that space, it is called Global Memory.

It stores the large data, ex) We copy the whole array of pixel values to the Global Memory in GPU & have d_x as a pointer pointing to it first element.

(ii) Registers:

These hold values which the thread needs immediately, such as Pixel values, indices & intermediate Calculations.

Ex) during int newval = img[index] + 40;

* A thread reads img[index] from GPU's Memory System. (Global)

DATE / /

S M T W T F S

* Once that value is read from global memory, Register stores a copy of it.
* Now, when value + brightness happens
  Ex) value = 100 → stored in register
  brightness = 40 → stored in Register
Now, ALU of streamly Multiprocessor will perform 100+40=140.

↓

Result written to another Register

↓

Result is stored back in the Global Mem.

Local Memory:
It is a conceptual Memory for the threads, backed by device Memory. It is more expensive so each thread could have it.

Image Buffer:
Place that stores the image pixel values. For now, such place is the

DATE / /

S M T W T F S

Global Memory: which stores the array of pixel values.

Cache: Keeps copies of data from Memory that the GPU might need again.

If any other thread needs same data, it first checks the cache, if present uses it, else, fetch from global Memory

This process of searching for the data in cache & data is available in cache

⇓

Cache Hit.

Process of searching for the data in Cache & data is not in cache

⇓

Cache Miss.

DATE / /

S M T W T F S
O O O O O O O

There are two types of cache:

* L1 cache
* L2 cache.

L1 cache	L2 cache
→ Closest to processing Resources → small & lower latency	→ Shared by GPU SMs → Larger than L1

If thread requests data

Check in L1 cache

↓ if cache miss

GPU tries to find in

L2 cache

↓ if not present

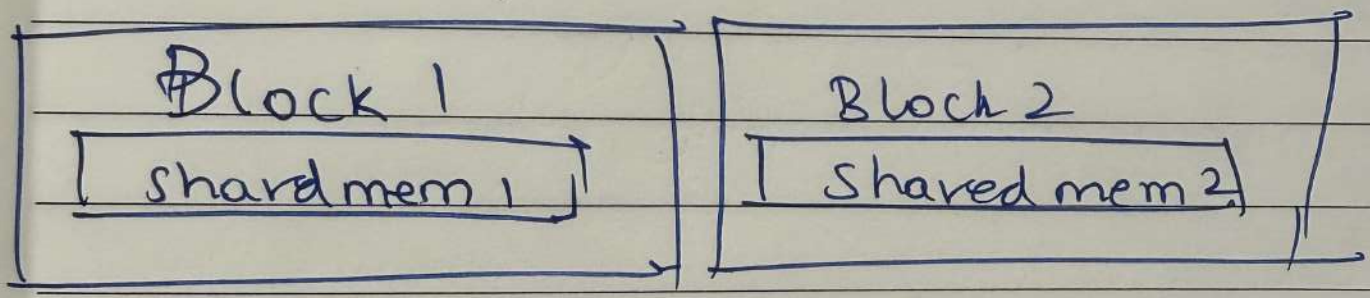
Global memory.

DATE / /

S M T W T F S
O O O O O O O

Shared Memory:

It is a Memory present for each Blocks, in which, threads only in that Block can access.



This helps greatly where there are some pixel overlaps, Ex)

Thread 1, 2, 3 inside Block 1 needs

1 → 10, 20, 30
2 → 20, 30, 40
3 → 30, 40, 50

} Pixel values

↓ overlap b/w the pixel vals.

So, it can use the shared Memory inside the Block.